Design and Implementation of a Multi-Tiered Serverless Pre-warming Simulator Under Concurrent Load Using Go and Docker

Thesis submitted in partial fulfillment of the requirements
for the degree of

B. Tech

in

Computer Science and Engineering

by

Farhan Javed

90/BCS/220012

Under the supervision of

Dr. Debabrata Sarddar

Department of Computer Science and Engineering
University of Kalyani
Kalyani, India

June, 2026

Declaration

I, Farhan Javed, declare that this thesis "Design and Implementation of a Multi-Tiered Serverless Pre-warming Simulator under Concurrent Load using Go and Docker" is a bona fide record of the work carried out by me under the supervision of Dr. Debabrata Sarddar. This report represents my ideas in my own words and where others' ideas or words have been included, I have adequately cited and referenced the original sources. I also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea, data, fact, or source in my submission. I further declare that the work reported in this thesis has not been and will not be submitted, either in part or in full, for the award of any other degree or diploma of this institute or any other institute. I understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Date: 25 June, 2026

Farhan Javed
90/BCS/220012

Acknowledgment

I want to take this opportunity to thank my supervisor, Dr. Debabrata Sarddar, Assistant Professor, Department of Computer Science and Engineering, University of Kalyani, for his motivation and tireless efforts in helping me gain a deep understanding of the research area and for supporting me throughout the life cycle of my B.Tech. course. Especially, the extensive comments, healthy discussions, and fruitful interactions with the supervisors had a direct impact on the final form and quality of this dissertation work.

I am also thankful to Dr. Priya Ranjan Sinha Mahapatra, Professor and Head of the Department of Computer Science and Engineering, University of Kalyani, for his fruitful guidance through the early years of chaos and confusion. I extend my heartfelt thanks to the faculty members and supporting staff of the Department of Computer Science and Engineering for their generous support, guidance, and unwavering cooperation throughout this journey.

This thesis would not have been possible without the unwavering support of my friends. I am deeply grateful to my parents for their blessings, love, and constant encouragement.

Certificate

This is to certify that the thesis entitled "Design and Implementation of a Multi-Tiered Serverless Pre-warming Simulator under Concurrent Load using Go and Docker" submitted by "Farhan Javed" (Reg. No. 1090117 of 2022-23, Roll No. 90/BCS/220012) in partial fulfillment for the award of B.Tech in Computer Science and Engineering is a bona fide work carried out under my supervision. The thesis fulfills the requirements as per the regulations of this University and, in my opinion, meets the necessary standards for submission. The contents of this thesis have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma, and the same is certified. It is understood that by this approval, the undersigned does not necessarily endorse any of the statements made or opinions expressed therein but approves it only for the purpose for which it is submitted.

Internal Supervisor External Supervisor Head of the Department

External Examiner(s)

Abstract

Serverless computing has become an attractive execution model because it removes infrastructure management from developers and provides pay-per-use semantics. However, the model suffers from a fundamental systems challenge: cold start latency. When no suitable execution environment is immediately available, a Function-as-a-Service (FaaS) platform must create a new runtime, initialize the language environment, and load application state before user code can begin. This initialization delay directly degrades responsiveness and is especially visible in bursty or intermittent workloads. Existing mitigation approaches typically make an unfavorable trade-off: either keep function environments permanently warm and pay the continuous memory cost, or rely on snapshots and restore mechanisms that may introduce storage or orchestration overhead.

This thesis presents the design and implementation of a **Multi-Tiered Serverless Prewarming Simulator** inspired by the multi-level readiness ideas described in recent systems research on AFaaS. Rather than reproducing kernel-level memory forking or hypervisor modifications, the project develops an application-level experimental simulator using Go and the Docker Engine API. The simulator models three readiness tiers: a cold tier that creates a fresh container on demand, a warm tier that reuses a long-lived running container, and a hot tier that maintains a paused container that can be rapidly resumed. A background eviction worker enforces a scale-to-zero policy by removing idle warm containers after a timeout, thereby simulating the tension between latency optimization and memory cost reduction.

The implemented system exposes an HTTP gateway for invocation requests, manages Docker containers across three tiers, logs per-request latency and active pool size to a CSV file, and generates plots that visualize both latency reduction and cost-oriented pool contraction. The work contributes a practical, explainable, and reproducible artifact suitable for educational analysis of FaaS performance trade-offs. It also frames cost not as an afterthought but as a first-class design concern by linking container residency to resource occupancy and by showing why aggressive pre-warming without eviction is economically unsound.

List of Abbreviations

AFaaS	Ant FaaS, research work motivating multi-level readiness
API	Application Programming Interface
CoW	Copy-on-Write
CPU	Central Processing Unit
CSV	Comma-Separated Values
FaaS	Function as a Service
Go	Go programming language
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
OSDI	Operating Systems Design and Implementation
RAM	Random Access Memory
RQ	Research Question
SDK	Software Development Kit
VM	Virtual Machine

List of Symbols

L_0	Cold start latency for Tier 0
L_1	Warm execution latency for Tier 1
L_2	Hot resume-and-execute latency for Tier 2
$C(t)$	Cost proxy at time t
$N_1(t)$	Number of resident Tier 1 containers at time t
$N_2(t)$	Number of resident Tier 2 containers at time t
T_e	Idle eviction threshold

Contents

1 Introduction

Serverless computing has shifted the unit of deployment and billing from the server to the function. The developer writes a small unit of logic, uploads it to a managed platform, and expects the infrastructure to provision execution environments on demand. This model simplifies deployment and offers excellent elasticity. Yet the abstraction comes with a hidden systems problem: when no ready execution context exists, the platform must pay an initialization penalty before the function can respond. This phenomenon is commonly referred to as the **cold start problem**.

1.1 Background and motivation

The cold start problem emerges because execution environments are not free. A FaaS platform must decide how much runtime state to keep resident in memory before a request arrives. If it keeps nothing warm, it pays high latency during bursts and after idle periods. If it keeps too much state warm, it consumes memory continuously even when requests stop arriving. Therefore, modern serverless systems face a core trade-off:

> Lower latency requires maintaining readiness in memory, while lower cost requires aggressively reclaiming idle resources.

Recent systems research, including the OSDI '25 AFaaS work, explores multi-level memory forking and tree-structured seeds to create several stages of readiness. That research operates much deeper in the systems stack and requires mechanisms close to the operating system, container runtime, or virtualization layer. The present work therefore adopts the **idea** rather than the exact implementation mechanism: instead of kernel-assisted memory seeds, it simulates levels of readiness using different Docker container states.

1.2 Problem statement

The problem addressed in this thesis is the following:

> How can one construct a practical simulator that demonstrates the latency-cost trade-off of serverless pre-warming using only application-level mechanisms, while remaining simple enough to implement, test, explain, and visualize clearly?

This question has both a technical and an educational dimension. Technically, the simulator must expose realistic enough behavior to reveal differences between cold, warm, and hot execution paths. Educationally, the system must remain understandable: every tier should map cleanly to a concrete Docker operation, and every design decision should be explainable in a viva or report defense.

1.3 Research objectives

The project has the following objectives:

1. Design a three-tier simulator for serverless invocation using Docker containers.
2. Build an HTTP gateway that accepts invocation requests and dispatches them to the warmest available tier.
3. Quantify latency across the three tiers through structured CSV logging.
4. Introduce a background eviction policy to simulate scale-to-zero behavior and cost control.
5. Stress-test the scheduler under concurrent load and summarize throughput, percentiles, and tier selection behavior.
6. Generate report-friendly visualizations that show latency benefit, memory-cost pressure, and load-test behavior.
7. Produce a codebase and thesis narrative that explain the system clearly enough for academic evaluation.

1.4 Why cost must be treated as a first-class concern

Performance optimization is often discussed in isolation. This is misleading in the context of serverless systems. Warm containers occupy memory, consume scheduler resources, and keep the host prepared for future work. Even when CPU consumption is near zero, retained runtime state still incurs cost. A pre-warmed system that never evicts stale state is simply shifting latency into a hidden memory bill. Therefore this thesis does not treat cost as a secondary metric. Instead, cost is explicitly represented through the active warm-container pool and through a scale-to-zero eviction worker.

The core thesis of the project is that **a useful serverless platform is not the one that always minimizes latency, but the one that manages latency and cost together**. This balance is the main reason a multi-tier design is preferable to a binary cold-versus-warm design.

2 Literature Review and Conceptual Basis

2.1 Cold starts in serverless platforms

Cold starts have been studied extensively because they directly affect user-facing response time. The initialization path of a serverless function typically includes container or microVM creation, runtime startup, dependency loading, application initialization, and user-code entry. In practice, the latency contribution of these steps depends on the language, platform design, host state, storage path, and concurrency level. Although the absolute numbers vary, the structural observation remains stable: reusing a more ready execution environment reduces startup delay.

2.2 Pre-warming and snapshot-based mitigations

Common cold start mitigation strategies include:

- **Static pre-warming:** Keep a pool of idle workers ready in advance.
- **Snapshot / checkpoint restore:** Resume from a previously captured runtime image.
- **Language-runtime caching:** Keep the interpreter or VM alive while refreshing only user code.
- **Predictive scaling:** Forecast demand and prepare instances before traffic bursts.

Each approach pays for latency reduction in a different currency. Static pre-warming spends memory continuously. Snapshot methods reduce startup work but may incur disk or orchestration overhead. Predictive scaling introduces model uncertainty. The academic value of the present simulator is that it makes this trade-off visible using only three easily understood states: create, reuse, and resume.

2.3 Representative systems and open platforms

The broader literature shows that cold-start mitigation is studied across both provider-grade runtimes and open platforms. Work on AWS Lambda and Firecracker emphasizes lightweight isolation, fast startup, and the practical systems trade-offs involved in serving short-lived functions at scale. Open-source platforms such as Apache OpenWhisk and Knative are also important because they expose scheduling, autoscaling, and cold-start behavior in inspectable implementations. This thesis is closer to the latter category: it does not claim provider-scale optimization, but it does construct a transparent platform for observing lifecycle trade-offs directly.

2.4 Checkpoint, restore, and runtime reuse

Another major line of work asks how much initialization can be skipped by restoring a previously prepared execution state. Snapshot-based approaches, partially initialized runtimes, and checkpoint/restore mechanisms such as CRIU all motivate the idea that readiness is not binary. A function can be merely reusable, partially restored, or resumed from a hotter preserved state. The present simulator does not implement kernel-level snapshot/restore, but Tier 2 is explicitly intended as an application-level approximation of that more prepared execution state.

2.5 Workload characterization and measurement context

Large-scale empirical studies also show why cold-start optimization is difficult in practice. Request arrivals are bursty, many functions are invoked infrequently, and providers must balance user-visible latency against the cost of keeping execution environments resident. This observation justifies the mixed evaluation strategy used in this thesis: controlled forced-tier baselines are needed to isolate the effect of each readiness level, while burst and high-concurrency scenarios are needed to expose contention and warm-pool saturation.

2.6 AFaaS as academic inspiration

The AFaaS line of work is important because it argues that not all warmness is the same. A function environment may be warm at different levels: base image loaded, language runtime loaded, application state initialized, or fully paused in memory. In the original research direction, these readiness levels are made economically viable through kernel-level Copy-on-Write (CoW) memory techniques, which allow multiple execution contexts to share common memory pages efficiently. This idea motivates the current simulator directly. The simulator does not implement CoW memory forking, seed trees, or kernel modifications; however, it does preserve the essential insight that **readiness is hierarchical**. A platform should therefore maintain multiple readiness levels and route requests to the hottest feasible state rather than treating all misses equally.

2.7 Gap addressed by this project

Most industrial serverless platforms are not easy to inspect internally, and many academic prototypes are too low-level to reproduce with standard application-level tooling. The gap, therefore, is a practical simulator that:

- is implementable in a common programming language,
- uses standard Docker operations rather than research kernels,

- still demonstrates the main latency-cost trade-off,
- logs useful data for graphs, discussion, and thesis writing.

This project fills that gap with a reproducible, application-level artifact.

2.8 Mapping the academic idea to the implemented simulator

It is useful to state explicitly how the implemented system maps the research inspiration into a practical artifact. The mapping is shown conceptually below:

- **Research idea:** multi-level readiness through memory-based seeds.
  Implemented approximation: three container states with different degrees of readiness.
- **Research idea:** fast reuse of partially prepared execution environments.
  Implemented approximation: reuse of running or paused Docker containers.
- **Research idea:** memory-footprint reduction through kernel-level Copy-on-Write (CoW) page sharing.
  Implemented approximation: memory-footprint control through strict scale-to-zero idle eviction, so unused warm states do not remain resident indefinitely.
- **Research idea:** balancing readiness retention against resource pressure.
  Implemented approximation: idle eviction and active pool-size logging.

This mapping is important because it identifies the core system principle and reconstructs it in a form that is technically feasible with standard application-level tooling.

3 Proposed Architecture

3.1 Architectural overview

The implemented simulator consists of four principal components:

A. **FaaS Gateway** - an HTTP server implemented in Go.

B. **Tiered Container Pool Manager** - a Docker-aware runtime controller.

C. **Idle Eviction Manager** - a background worker enforcing scale-to-zero behavior.

D. **Metrics and Plotting Pipeline** - CSV logging followed by Python-based visualization.

3.2 Three-tier execution model

The core design choice is to represent readiness using three observable Docker states. Table 1 summarizes the mapping.

Tier	Name	Execution strategy	Expected trade-off
0	Cold Start	Create fresh container, run task, wait for exit, remove container	Highest startup cost, lowest idle memory cost
1	Warm Runtime	Keep one container running; use `docker exec` for each invocation	Lower startup cost, moderate idle memory cost
2	Hot Paused	Keep one container paused; unpause, execute, and pause again	Lowest startup cost among simulated tiers, highest readiness retention cost

Table 1: Comparison of the three simulated container tiers.

3.3 Routing policy

The routing policy is intentionally simple and deterministic. When a request arrives:

1. If Tier 2 exists and is not busy, use Tier 2.

2. Else if Tier 1 exists and is not busy, use Tier 1.

3. Else fall back to Tier 0.

This policy embodies the main systems idea of the thesis: **prefer the warmest ready container**. It is easy to explain, easy to test, and suitable for generating clean comparative results.

3.4 Cost-aware lifecycle design

The design explicitly avoids keeping warm resources alive forever. Both Tier 1 and Tier 2 record a `LastUsed` timestamp. A background worker periodically checks idle duration and removes containers whose idle time exceeds the eviction threshold. This translates the abstract concept of "paying for warmness" into a concrete implementation rule.

3.5 Request lifecycle from arrival to completion

The full lifecycle of one invocation is worth describing in detail because it ties together all components of the system:

1. A client sends an HTTP `POST` request to `/invoke`.
2. The gateway decodes the JSON body into an `InvokeRequest`.
3. The pool manager checks whether a tier was forced for experimental control.
4. If no tier is forced, the manager checks Tier 2 first, then Tier 1, then Tier 0.
5. Once a tier is selected, the simulator marks the tier as busy so concurrent requests do not reuse it unsafely.
6. The task executes through one of three paths:
    - Tier 0 creates a fresh container and waits for it to exit.
    - Tier 1 creates a Docker exec session inside a running container.
    - Tier 2 unpauses the container, executes the task, and pauses it again.
7. Latency is measured from just before execution until the task completes.
8. The system updates the container's last-used timestamp if a warm tier was involved.
9. The request result is written to `metrics.csv`.
10. A JSON response is sent back to the client.

This lifecycle is useful in the thesis because it makes clear that the simulator is not merely storing containers in the background; it is managing them as reusable execution assets with explicit state transitions and instrumentation.

3.6 Metrics pipeline

Each invocation produces a single metric record containing:

- timestamp,
- request identifier,
- tier used,
- total invocation latency in milliseconds,
- active container pool size.

The first four metrics capture performance. The last metric acts as a simple cost proxy: a larger warm pool suggests more resident state and therefore more ongoing memory burden.

4 Implementation Details

4.1 Project structure

The project is intentionally kept flat so that each source file corresponds to one major responsibility:

- `main.go` - bootstraps configuration, logger, pool manager, and HTTP server.
- `config.go` - environment-based configuration.
- `gateway.go` - HTTP request handling and JSON API surface.
- `pool.go` - all core container lifecycle logic.
- `metrics.go` - CSV metric persistence.
- `Dockerfile` and `task.py` - mock function runtime image.
- `plot.py` - graph generation from collected metrics.

4.2 Representative code snippets

This section presents selected excerpts from the implementation to illustrate the core design decisions. It does not reproduce the full repository; instead, it highlights the most relevant parts of the system.

4.3 Internal state management and concurrency reasoning

Even though the simulator is conceptually simple, it still has to manage mutable shared state safely. The pool manager stores pointers to the current Tier 1 and Tier 2 containers, tracks whether they are busy, records last-used timestamps, and generates request identifiers. Because requests can arrive concurrently, this state cannot be manipulated casually.

The implementation therefore uses:

- a mutex for protecting shared container state,
- explicit busy flags to prevent overlapping reuse of the same resident container,
- an atomic counter for generating unique request identifiers,
- background goroutines only where asynchronous behavior is logically justified.

4.3.1 Why busy flags are needed

If two requests attempted to use the same warm container at the same time without coordination, the simulator could produce race conditions or confusing results. For example, one request could unpause a Tier 2 container while another attempts to pause it again, or two `docker exec` calls could overlap in a way that makes latency interpretation less clean. The `Busy` flag prevents this by making each resident warm container a single-tenant execution resource in the current implementation.

This choice also improves experimental clarity. Since the thesis focuses on tier behavior rather than throughput optimization, it is better to serialize access to a single warm instance than to allow concurrent reuse that would complicate the analysis.

4.3.2 Configuration model

The system configuration is centralized in a single Go struct. This makes runtime parameters explicit and easy to tune during experiments.

```
type Config struct {
    Port           string
    MetricsFile    string
    ImageName      string
    TaskDurationMS int
    EvictionAfter  time.Duration
    SweepInterval  time.Duration
    RequestTimeout time.Duration
    DockerTimeout  time.Duration
    AutoWarmStart  bool
    WarmOnInvoke   bool
}
```

Listing 1: Configuration fields used by the simulator.

This configuration allows the simulator to be re-run under different eviction and task-duration settings without modifying code. In experimental work, this is useful because the thesis may later compare short and long idle thresholds.

4.3.3 Gateway request model

The HTTP API accepts a simple JSON body. This keeps the experimental interface small while still allowing controlled tests.

```go
type InvokeRequest struct {
    RequestID  string          `json:"request_id"`
    DurationMS int             `json:"duration_ms,omitempty"`
    ForceTier  *int            `json:"force_tier,omitempty"`
    Payload    json.RawMessage `json:"payload,omitempty"`
}
```

Listing 2: Invocation request model exposed by the FaaS gateway.

The `force_tier` field is especially useful for thesis experiments because it allows explicit comparison of Tier 0, Tier 1, and Tier 2 without waiting for the scheduler to choose them naturally.

4.3.4 Tier selection logic

The pool manager uses a warmest-available routing rule. A simplified version of the logic is shown below.

```go
if m.tier2 != nil && !m.tier2.Busy {
    return chooseTier(2, m.tier2)
}
if m.tier1 != nil && !m.tier1.Busy {
    return chooseTier(1, m.tier1)
}
return 0, nil, nil
```

Listing 3: Simplified tier acquisition policy.

This code fragment captures the core idea of the thesis in executable form: prefer the hottest reusable state, otherwise pay the cold-start cost.

4.3.5 Tier 2 resume path

Tier 2 is the most interesting path because it explicitly models a hot-but-not-active container. On each request the container is unpaused, the task is executed, and then the container is paused again.

```
if err := m.cli.ContainerUnpause(opCtx, ctr.ID); err != nil {
    return err
}

if err := m.execTaskInContainer(opCtx, ctr.ID, durationMS); err != nil {
    return err
}

if err := m.cli.ContainerPause(opCtx, ctr.ID); err != nil {
    return err
}
```

Listing 4: Tier 2 execution path.

In thesis terms, Tier 2 represents the strongest readiness retention in the simulator because the container remains instantiated and only has to resume execution before serving work.

4.3.6 Fallback behavior and robustness

Real systems must handle failed warm paths. The simulator therefore includes a fallback idea: if a warm tier fails during invocation and the request was not explicitly forced to that tier for experimental reasons, the system invalidates that warm tier and falls back to a Tier 0 cold start. This is important for two reasons.

First, it makes the gateway more robust during operation. A broken warm container does not necessarily bring the whole service down. Second, it reflects a realistic operational principle: warm-state optimization is a performance enhancement, not the only correctness path. The cold-start path remains the baseline mechanism that always preserves functional correctness.

4.3.7 Why the task is intentionally synthetic

The function workload used by the simulator is a simple sleep-based Python script. At first glance this may seem too simple, but it is a deliberate experimental choice. If a complex application workload were used, the measured latency would mix together multiple factors:

- application logic time,
- dependency loading time,
- language runtime behavior,
- Docker lifecycle overhead.

By using a synthetic sleep task, the experiment isolates the platform overhead more clearly. The goal is not to demonstrate business logic performance. The goal is to compare the cost of **getting to execution** under different readiness states.

4.4 Design decisions that were considered but not adopted

The final architecture is clearer when contrasted with alternatives that were intentionally rejected.

4.4.1 Why not maintain many warm containers per tier

An obvious extension would be to maintain multiple resident containers so the simulator could absorb concurrent traffic more realistically. This was not adopted in the current version because it would introduce a second research problem: pool scaling policy. Once multiple warm containers exist, the system must answer additional questions such as:

- How many warm containers should be kept alive?
- When should a new warm container be added?
- Which warm container should receive the next request?
- Which one should be evicted first?

Those are valuable questions, but they would dilute the thesis focus. The current implementation isolates the readiness hierarchy first, which is the more foundational contribution.

4.4.2 Why not implement prediction-based pre-warming

Prediction-based pre-warming is attractive because it attempts to prepare resources before traffic arrives. However, it introduces forecasting, workload history modeling, and false-positive/false-negative trade-offs. That would move the project toward machine learning or time-series prediction rather than systems design. Since the central thesis is about architectural readiness and cost trade-offs, deterministic warm-state management was the more defensible choice.

4.4.3 Why not use a real web application as the function

Using a real application inside the containers might make the system more visually appealing, but it would make the measured data harder to interpret. A realistic application introduces its own caches, initialization sequence, and dependency behavior. The resulting numbers

would say less about the container tiering policy and more about the chosen application. For this study, it is better to simplify the workload so the platform behavior remains the dominant variable.

4.4.4 Idle eviction worker

The cost-control mechanism is implemented as a background goroutine with a periodic ticker.

```
func (m *PoolManager) StartEvictionWorker(ctx context.Context) {
    ticker := time.NewTicker(m.cfg.SweepInterval)

    go func() {
        defer ticker.Stop()
        for {
            select {
            case <-ctx.Done():
                return
            case <-ticker.C:
                m.evictIdle(ctx)
            }
        }
    }()
}
```

Listing 5: Idle eviction worker.

This snippet is central to the cost argument of the project. Without it, warm containers would remain in memory indefinitely and the simulator would demonstrate only performance gain, not the cost-performance balance.

4.5 Mock execution environment

The Docker image intentionally contains only a minimal Python runtime and a small script that sleeps for a configurable duration. The goal is not to benchmark application logic, but to keep the workload stable so that differences in startup path remain visible.

```
FROM python:3.12-alpine

WORKDIR /app

COPY task.py /app/task.py

CMD ["sh", "-c", "while true; do sleep 3600; done"]
```

Listing 6: Dockerfile for the mock FaaS environment.

```python
import sys
import time

def main() -> None:
    sleep_ms = 1000
    if len(sys.argv) > 1:
        sleep_ms = int(sys.argv[1])
    time.sleep(sleep_ms / 1000.0)

if __name__ == "__main__":
    main()
```

Listing 7: Mock task executed inside the container.

4.6 Metrics logging implementation

Each invocation writes one row to `metrics.csv`. This makes the experiment transparent and easy to re-plot. The schema is deliberately simple:

```
Timestamp,RequestID,TierUsed,LatencyMS,ActiveContainersPoolSize
```

Listing 8: CSV columns emitted by the simulator.

For high-concurrency experiments, a companion Go load generator writes one summary row per scenario to `loadtest_summary.csv`. This second file records achieved throughput, success and error counts, latency percentiles, and the number of responses served by each tier. Together, the two CSV files provide both request-level detail and scenario-level performance summaries.

5 Experimental Methodology

5.1 Experimental questions

The implementation is designed to answer the following practical questions:

RQ1: Does a warmer execution tier reduce invocation latency?

RQ2: Does idle eviction reduce the size of the resident pool during inactive periods?

RQ3: Can the active pool size act as a simple cost proxy in an application-level simulator?

RQ4: Does the scheduler remain stable and interpretable under sustained concurrent load?

5.2 Experimental environment

The simulator is intended to be run on a developer workstation with Docker installed. On macOS, this usually means Docker Desktop must be running because the Docker daemon is provided through that environment. On Arch Linux, the Docker daemon can be run directly through `systemd`. The application itself does not change across platforms; only the host Docker setup differs.

The experimental workflow is:

1. Build the mock image.
2. Start the Go server.
3. Send controlled invocations and load-test scenarios using the built-in Go client.
4. Observe log entries in `metrics.csv` and scenario summaries in `loadtest_summary.csv`.
5. Run `plot.py` to generate report figures.

5.3 Hardware and software environment

The measurements discussed later were collected on a single local workstation. Because latency values depend strongly on the host configuration, the relevant platform details are summarized in Table 2.

Component	Value
CPU	Intel(R) Core(TM) i7-9750H CPU @ 2.60GHz
RAM	16 GB system memory (approximately 15.5 GiB visible to the OS)
OS	Arch Linux, kernel 7.0.12-arch1-1, x86_64
Docker Version	Docker client/server 29.6.0
Go Version	go1.26.4-X:nodwarf5 linux/amd64

Table 2: Hardware and software environment used for local experimentation.

5.4 Suggested workload scenarios for a richer thesis evaluation

To increase the depth of the final evaluation chapter, the simulator can be tested under several traffic patterns instead of only a single invocation sequence.

5.4.1 Scenario A: Pure cold-start baseline

Force every invocation to Tier 0. This establishes the worst-case startup path and gives a clean baseline against which the warm tiers can be compared.

5.4.2 Scenario B: Warm-tier comparison

Force several requests to Tier 1 and then Tier 2 separately. This helps quantify how much latency is saved by keeping a container running versus keeping it paused and resumable.

5.4.3 Scenario C: Burst followed by idle period

Send a short burst of requests, then wait longer than the eviction threshold, and finally send another request. This scenario is particularly important because it demonstrates both sides of the design:

- during the burst, the system benefits from warm reuse;
- after idle time, the system should reclaim warm resources.

5.4.4 Scenario D: Mixed automatic routing

Do not force any tier. Let the scheduler choose the warmest available option and observe how the natural routing behavior aligns with the intended policy.

5.4.5 Scenario E: High-concurrency load test

Run the simulator under a sustained request rate using the dedicated Go load generator. A representative stress scenario is 1,000 requests per second for a short fixed interval. This scenario does not aim to prove production-scale throughput for Docker itself; instead, it tests whether the gateway, mutex-protected pool state, and fallback logic remain thread-safe and measurable under heavy parallel arrival pressure.

Including these scenarios in the report strengthens the evaluation by turning it from a single example into a small experimental study.

5.5 Execution procedure

Representative commands are shown below.

```
go mod tidy
docker build -t multitier-faas-mock:latest .
go run .
```

Listing 9: Commands used to build and run the simulator.

```
curl -X POST http://localhost:8080/invoke \
  -H "Content-Type: application/json" \
  -d '{"request_id":"cold-1","force_tier":0}'

curl -X POST http://localhost:8080/invoke \
  -H "Content-Type: application/json" \
  -d '{"request_id":"warm-1","force_tier":1}'

curl -X POST http://localhost:8080/invoke \
  -H "Content-Type: application/json" \
  -d '{"request_id":"hot-1","force_tier":2}'
```

Listing 10: Controlled invocation commands for each tier.

```
go run ./cmd/loadtest \
  -label cold-baseline \
  -force-tier 0 \
  -total 20 \
  -rps 5 \
  -concurrency 1

go run ./cmd/loadtest \
  -label auto-burst \
  -total 200 \
  -rps 100 \
  -concurrency 32
```

```
go run ./cmd/loadtest \
  -label high-concurrency \
  -seconds 3 \
  -rps 1000 \
  -concurrency 128 \
  -duration-ms 50
```

Listing 11: Repeatable load-test commands using the built-in Go client.

```
./scripts/run_thesis_workload.sh
```

Listing 12: One-command thesis workload for baseline, burst, eviction, and stress tests.

5.6 Expected output examples

When the system is functioning correctly, the HTTP response should return the request identifier, chosen tier, latency, and pool size. A representative response is shown below.

```
{
  "request_id": "hot-1",
  "tier_used": 2,
  "latency_ms": 12.731,
  "active_containers_pool_size": 2,
  "message": "invocation completed"
}
```

Listing 13: Example JSON response from the gateway.

Similarly, a representative `metrics.csv` extract may look as follows:

```
Timestamp,RequestID,TierUsed,LatencyMS,ActiveContainersPoolSize
2026-02-15T10:00:01.245Z,cold-1,0,1187.420,2
2026-02-15T10:00:05.512Z,warm-1,1,47.883,2
2026-02-15T10:00:09.771Z,hot-1,2,12.731,2
```

Listing 14: Example `metrics.csv` output.

```
RunLabel,Success,Errors,ObservedRPS,AvgLatencyMS,P95LatencyMS,P99LatencyMS,
    Tier0Count,Tier1Count,Tier2Count
cold-baseline,20,0,4.99,1214.310,1242.118,1251.770,20,0,0
high-concurrency,3000,0,987.41,287.114,591.442,710.885,2879,61,60
```

Listing 15: Example load-test summary output.

6 Cost Model and Design Reasoning

6.1 Why active pool size is used as a cost proxy

In a production serverless platform, cost would be modeled more precisely through RAM occupancy, CPU reservations, storage footprint, orchestration overhead, and provider billing rules. The present simulator does not attempt to infer those values exactly because the goal is not billing accuracy. Instead, it uses `ActiveContainersPoolSize` as an interpretable proxy for memory retention.

This decision is justified for three reasons:

1. The primary cost of pre-warming is residency: containers kept alive or paused continue to occupy host resources.
2. Active pool size increases when warm containers exist and decreases when eviction reclaims them; therefore it tracks the direction of cost pressure even if it does not express exact currency.
3. A simple metric is easier to justify and interpret than a pseudo-precise but weakly justified billing formula.

6.2 A deeper view of cost in serverless systems

Cost in serverless systems is often misunderstood because the end user is typically shown a simple per-request billing abstraction. Internally, however, a platform operator must pay for several layers of readiness:

- memory retained by idle runtimes,
- CPU scheduling overhead for resident workers,
- metadata and orchestration state,
- storage or image cache state,
- capacity reserved to absorb bursts.

The present simulator does not monetize all of these layers in currency, but it does model their most visible educational consequence: **retained resident state has a cost even when no requests are arriving**. This is why Tier 1 and Tier 2 are not free optimizations. They improve latency by reserving future execution readiness in advance.

This cost reasoning also explains why eviction is modeled for both Tier 1 and Tier 2. In a naive design, one might argue that a single warm container is too small to worry about. That misses the systems principle. Even one resident worker represents a standing allocation decision. In a real multi-tenant platform serving many functions, the difference between evicted and non-evicted warm pools becomes financially significant very quickly.

6.3 A conceptual cost-proxy formulation

Let the cost proxy at time t be:

$$C(t) = \alpha N_1(t) + \beta N_2(t)$$

where:

- $N_1(t)$ is the number of resident Tier 1 containers,
- $N_2(t)$ is the number of resident Tier 2 containers,
- α and β are weighting constants,
- $\beta > \alpha$ if Tier 2 is considered costlier because it preserves a hotter state.

In the implemented artifact, this expression is simplified operationally by logging the total active resident pool size:

$$ActiveContainersPoolSize = N_1(t) + N_2(t)$$

This simplification is acceptable for the current thesis because the project focuses on the existence of the trade-off rather than the exact commercial billing model. Accordingly, the expression should be interpreted as a conceptual cost proxy, not as a provider-accurate billing equation.

6.4 Why scale-to-zero matters

If a simulator only keeps warm containers and never removes them, the evaluation becomes biased. It would prove only that preserving state reduces latency, which is obvious. The more interesting systems question is whether the platform can **recover cost during idle periods**. Scale-to-zero is therefore not an optional feature in this project; it is necessary to express the cost side of the trade-off.

The eviction threshold T_e determines the system's posture:

- small T_e means aggressive cost saving but more future cold starts,
- large T_e means lower future latency but more time spent holding memory.

This is exactly the kind of engineering compromise real FaaS platforms must navigate.

6.5 Reasoning behind the three tiers

The project could have been implemented with only two states: cold and warm. However, this would miss a major insight from the motivating literature: readiness is not binary. There is value in distinguishing a running warm environment from a paused hot environment. The three-tier structure gives the thesis richer interpretive power:

- Tier 0 shows full setup cost.
- Tier 1 isolates the benefit of reusing a running language environment.
- Tier 2 isolates the additional benefit of preserving an even hotter state between requests.

This makes the resulting graphs more meaningful and helps explain why hierarchical prewarming is attractive.

6.6 Why Tier 2 is conceptually important

Tier 2 deserves special emphasis because it makes the thesis more than a simple cold-versus-warm story. Without Tier 2, the simulator would show that container reuse is faster than fresh creation. That is useful, but limited. Tier 2 introduces a more subtle question: if a runtime has already been prepared, can it be preserved in a low-activity state that still allows faster future activation than a general warm pool?

This is important because it reflects the broader systems insight that there are multiple levels of "warmness." A paused container is not actively serving requests, but it is also not gone. It sits in an intermediate state between active service and full destruction. That intermediate state is what makes AFaaS-like ideas intellectually interesting, and including Tier 2 in the simulator gives the thesis a stronger conceptual connection to the motivating research.

7 Results and Discussion

7.1 Expected qualitative results

The expected latency trend of the simulator is that reused execution environments should outperform fresh cold starts:

$$\min(L_1, L_2) < L_0$$

The reasoning is straightforward:

- Tier 0 performs the most work before user code begins because it must create and start a new container.
- Tier 1 avoids container creation and instead injects work into an already running environment.
- Tier 2 preserves a hotter paused state, but in a Docker-based implementation the additional unpause and re-pause operations may or may not outperform Tier 1 on every host. Therefore the important claim is that both reused tiers should remain below Tier 0 on average, while the relative ordering of Tier 1 and Tier 2 may vary.

The expected pool-size behavior is:

- During bursty traffic, the active pool size remains above zero because warm state is maintained.
- After prolonged inactivity, the pool size should drop to zero due to idle eviction.

For the high-concurrency scenario, the expected behavior is more nuanced. Because the current implementation maintains only one resident container for Tier 1 and one for Tier 2, a large incoming wave will quickly consume both warm slots. Additional concurrent requests should therefore fall back to Tier 0. This does not represent a scheduling bug; it reflects the intentionally small warm pool. The stress test is valuable because it shows that the system remains correct and measurable even when demand exceeds warm capacity.

7.2 How to reason about surprising or imperfect results

Real measurements may not always appear perfectly ordered in every individual sample. For example, a particular Tier 1 invocation might look slower than expected because the

host machine is under load, Docker is performing background work, or the operating system scheduler introduces noise. This does not invalidate the thesis. It simply means the discussion should focus on **aggregate trends** rather than isolated outliers.

The correct academic approach is:

1. report the data honestly,
2. identify the dominant trend,
3. acknowledge possible sources of noise,
4. explain why the trend still supports the architectural argument.

This is an important point in the thesis because technical evaluations are stronger when they discuss uncertainty directly rather than pretending the experiment is noiseless.

7.3 Experimental findings and interpretation

The recorded measurements, as visualized in the accompanying figures, provide evidence supporting the thesis hypothesis while also revealing an important implementation-specific effect of Docker control-path overhead:

1. **Latency comparison and control-path overhead.** The controlled baseline comparison, derived from 12 forced invocations per tier, shows the expected first-order trend: the cold-start path (Tier 0) is the slowest path, while both reused tiers reduce mean latency. In the recorded run, Tier 1 appears lower than Tier 2 even though Tier 2 is conceptually hotter. In this simulator, Tier 2 requires three Docker control operations (`unpause`, `exec`, and `pause`), whereas Tier 1 requires only a single `exec` path into an already running container. The results therefore suggest that control-path overhead can outweigh the theoretical readiness advantage of the paused state for a lightweight synthetic workload. This interpretation is consistent with the broader AFaaS argument that execution readiness alone does not determine end-to-end latency; the interface and orchestration path also matter.
2. **Cost-pressure behavior.** The active container pool-size graph confirms the economic discipline of the simulator. During request bursts, the system maintains a pool size of 2, preserving resident warm state for speed. After the configured idle threshold is exceeded, the eviction worker reclaims these resources and the pool size drops to zero before later being rebuilt. This demonstrates that the system does not retain warm memory indefinitely, but enforces a scale-to-zero model that makes the latency-cost trade-off visible.

3. **Behavior under high concurrency.** The load-test summary and latency-over-time plots illustrate the system's behavior under burst contention. While isolated requests are handled efficiently by the warm tiers, the high-concurrency scenario quickly saturates the single-instance warm containers. Overflow traffic is then routed to Tier 0, and the summary plot shows that tail latency rises into the multi-second range. This result does not imply scheduler failure; rather, it shows the consequence of limited warm-pool capacity under heavy load. The observed behavior is consistent with the AFaaS literature's emphasis on contention and orchestration overhead as major contributors to end-to-end serverless latency.

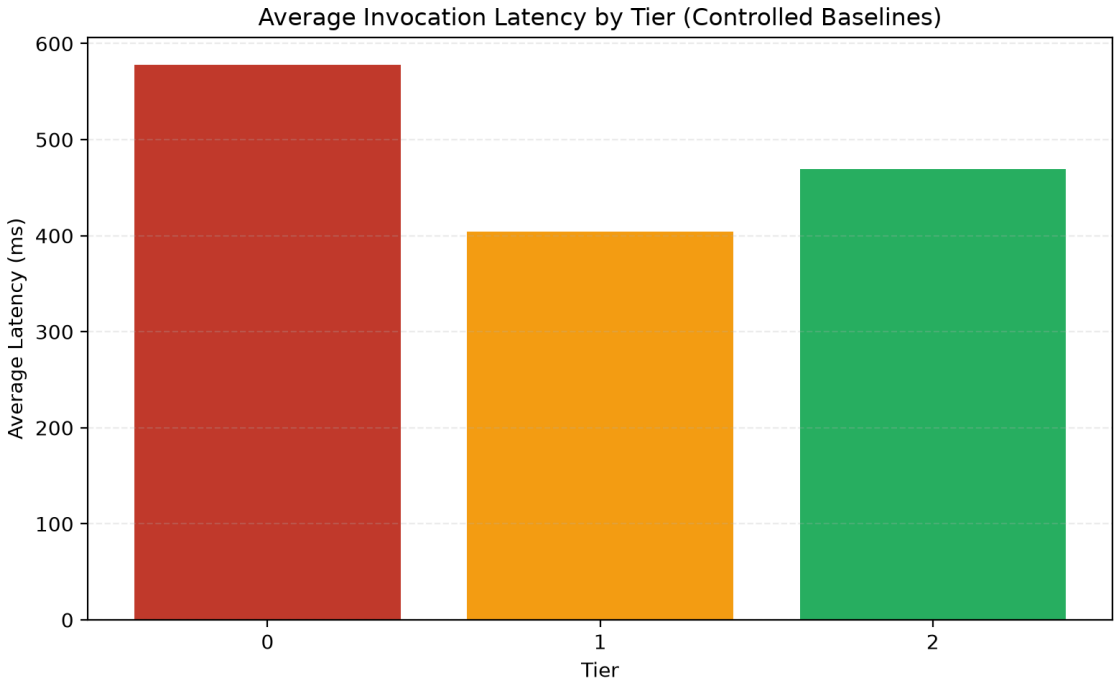

Figure 1: Average latency across Tier 0, Tier 1, and Tier 2 for the controlled baseline runs only.

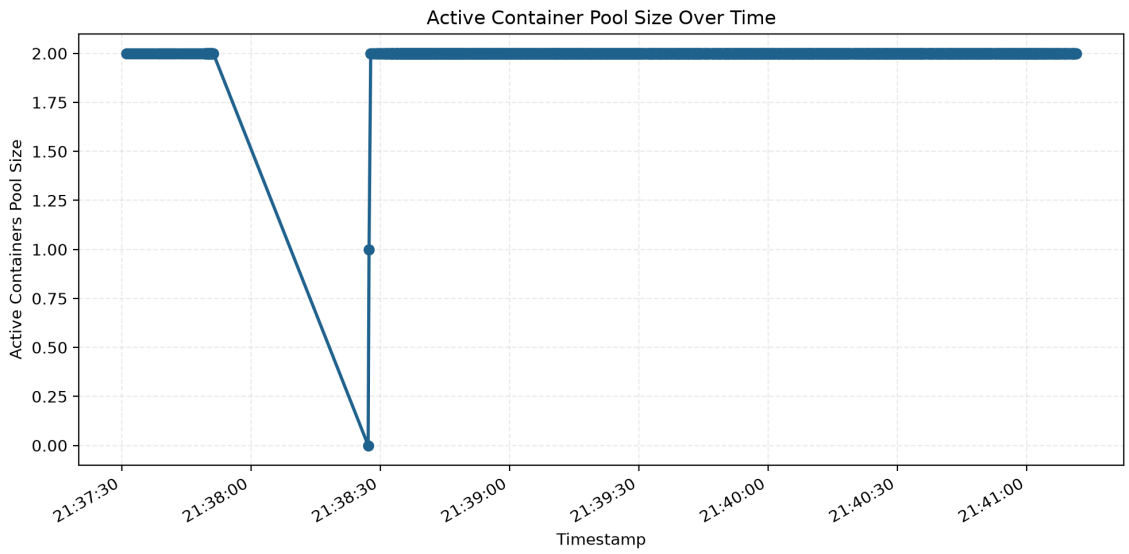

Figure 2: Container pool size over time, showing warm retention during bursts and contraction after idle eviction.

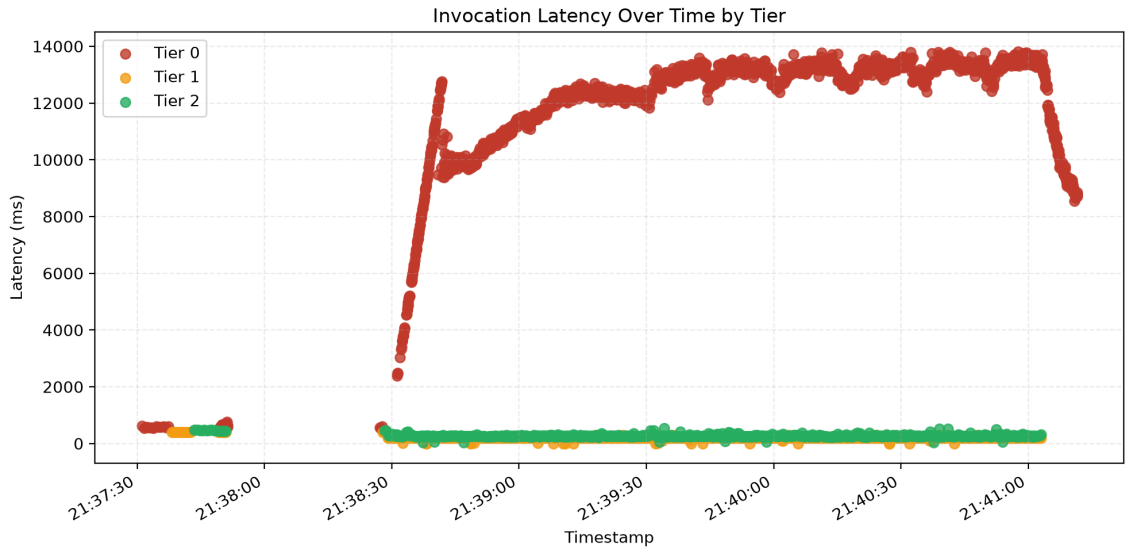

Figure 3: Invocation latency over time colored by selected tier.

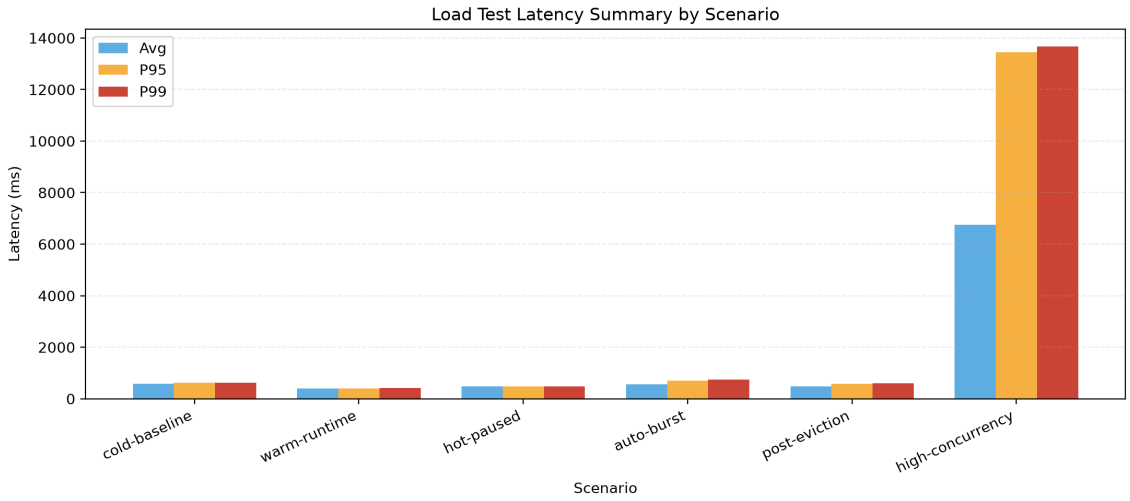

Figure 4: Average, P95, and P99 latency for each load-test scenario captured in `loadtest_summary.csv`.

7.4 Interpretation of the generated plots

Bar chart interpretation. The average-latency bar chart should be read as a controlled baseline comparison. It is generated from the tier-forced scenarios only, so it isolates the relative cost of Tier 0, Tier 1, and Tier 2 without mixing in the high-concurrency stress run. If the reused tiers remain below Tier 0, the simulator has demonstrated the expected benefit of keeping execution environments ready.

Why Tier 1 may outperform Tier 2. In the present implementation, Tier 2 is conceptually hotter because the container remains preserved in a paused state. However, each Tier 2 invocation must still traverse a longer Docker control path: the gateway unpauses the container, executes the task, and pauses it again. Tier 1 requires only a single `docker exec` path into an already running container. As a result, Docker daemon and API overhead can outweigh the theoretical readiness advantage of Tier 2 for a lightweight synthetic workload. This does not contradict the broader thesis. It instead shows that execution readiness and control-path overhead must be analyzed together.

Line graph interpretation. The active-pool-size graph should show two distinct phases. During request bursts, the line should remain elevated because resident warm containers are maintained. After an idle period that exceeds the configured eviction threshold, the line should drop, demonstrating that the system is reclaiming resources instead of holding them indefinitely.

Latency-over-time interpretation. The tier-colored latency plot is useful for separating two effects that a bar chart alone cannot show: how the scheduler switches among tiers over time, and how the post-eviction request returns to a colder path. Warm and hot requests should cluster lower on the graph, while Tier 0 points should appear noticeably higher.

Load-summary interpretation. The grouped latency plot derived from `loadtest_summary.csv` supports the concurrency discussion. It should show that controlled warm-path scenarios have lower latency than cold baselines, while the high-concurrency scenario produces much wider tail latency because the single warm instances are quickly saturated and many requests are forced back to Tier 0. If the observed throughput is far below the target request rate, that should be discussed explicitly as evidence of contention and limited warm-pool capacity rather than hidden or rounded away.

7.5 What this simulator supports and what it does not establish

The simulator shows that an application-level system can visibly demonstrate the major serverless trade-off between latency and retained warm state. It also shows that multi-level readiness is a more informative design than a simple warm/cold binary. However, the simulator does not claim to reproduce the exact microsecond-level or memory-sharing behavior of kernel-assisted systems such as those studied in advanced research prototypes. Its contribution is conceptual clarity, reproducibility, and measurable behavior in a practical development environment.

7.6 Limitations

The present study has several limitations that should be stated explicitly:

- Docker containers are used as the execution substrate instead of a production FaaS runtime such as AWS Lambda, OpenWhisk, or Knative Serving.
- The simulator does not implement kernel-level memory sharing, Copy-on-Write seed trees, or full snapshot/restore integration.
- Only one resident container is maintained for Tier 1 and one for Tier 2, so the warm pool is intentionally small.
- The workload is synthetic and sleep-based, which isolates control-path effects but does not represent complex application initialization.
- The reported latency figures are therefore useful for within-system comparison, but they should not be interpreted as directly comparable to production cloud provider measurements.

8 Advantages and Future Work

8.1 Advantages of the implemented design

The implemented simulator offers several strengths:

- It maps complex systems ideas onto familiar Docker operations.
- It remains small enough for full code comprehension.
- It produces concrete output artifacts such as CSV logs and plots.
- It embeds cost thinking directly through eviction and pool-size tracking.
- It now includes a repeatable stress-testing path for concurrency and scheduler validation.
- It can be presented and evaluated directly from observable runtime behavior and generated artifacts.

8.2 Future work

Several extensions would make strong future work items:

1. Support multiple warm containers per tier and implement queue-aware routing.
2. Measure real memory usage of each resident container and incorporate it into the cost model.
3. Add workload prediction so warm pools can expand before bursts.
4. Compare pause-based resumption with snapshot-restore approaches.
5. Introduce multiple mock functions with different runtimes and startup profiles.
6. Package the simulator with a web dashboard for real-time visualization.

9 Conclusion

This thesis presented the design and implementation of a Multi-Tiered Serverless Prewarming Simulator under Concurrent Load built with Go and Docker. The system was inspired by the multi-level readiness concepts associated with AFaaS research, but intentionally implemented at the application level in order to remain feasible, reproducible, and explainable with standard tooling. The simulator exposes three execution tiers - cold, warm, and hot paused - and routes incoming requests to the warmest available container. By logging latency and active pool size and by reclaiming idle warm containers through an eviction worker, the system makes the central serverless trade-off visible: lower latency requires retained readiness, while lower cost requires aggressive reclamation of idle state.

The project demonstrates that even without kernel-level support, a carefully designed simulator can communicate the core ideas behind modern serverless optimization research. It also shows that performance must not be discussed independently of cost. A system that is always warm may be fast, but it is not economically disciplined. The scale-to-zero eviction logic is therefore as important to the thesis as the low-latency execution paths themselves. Taken together, the implementation, metrics pipeline, and report structure provide a coherent artifact suitable for technical analysis and academic discussion.

Appendix A: Additional Code Snippets

This appendix presents selected implementation excerpts for reference. Two examples are given below.

```
type MetricRecord struct {
    Timestamp                time.Time
    RequestID                string
    TierUsed                 int
    LatencyMS                float64
    ActiveContainersPoolSize int
}
```

Listing 16: Metrics record used for CSV output.

```
df = pd.read_csv(metrics_path)
df["Timestamp"] = pd.to_datetime(df["Timestamp"])
request_df = df[df["TierUsed"].isin([0, 1, 2])].copy()

baseline_df = request_df[
    request_df["RequestID"].astype(str).str.startswith(
        ("cold-baseline-", "warm-runtime-", "hot-paused-")
    )
].copy()

latency = baseline_df.groupby("TierUsed", as_index=False)["LatencyMS"].mean()
plt.bar(latency["TierUsed"].astype(str), latency["LatencyMS"])

plt.plot(request_df["Timestamp"], request_df["ActiveContainersPoolSize"])
plt.scatter(request_df["Timestamp"], request_df["LatencyMS"])
```

Listing 17: Core plotting logic in `plot.py`.

Bibliography

1. Xiaohu Chai et al. "Fork in the Road: Reflections and Optimizations for Cold Start Latency in Production Serverless Systems." In *Proceedings of the 19th USENIX Symposium on Operating Systems Design and Implementation (OSDI '25)*, 2025. Available at: `https://www.usenix.org/system/files/osdi25-chai-xiaohu.pdf`

2. Alexandru Agache et al. "Firecracker: Lightweight Virtualization for Serverless Applications." In *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, 2020. Available at: `https://www.usenix.org/system/files/nsdi20-paper-agache.pdf`

3. Muhammed Golec et al. "Cold Start Latency in Serverless Computing: A Systematic Review, Taxonomy, and Future Directions." *ACM Computing Surveys*, 2025. Available at: `https://dl.acm.org/doi/10.1145/3700875`

4. Johann Schleier-Smith et al. "What Serverless Computing Is and Should Become: The Next Phase of Cloud Computing." *Communications of the ACM*, 64(5), 2021. Available at: `https://dl.acm.org/doi/10.1145/3406011`

5. Nima Kaviani et al. "Towards Serverless as Commodity: a case of Knative." In *Proceedings of the 5th International Workshop on Serverless Computing (WoSC '19)*, 2019. Available at: `https://dl.acm.org/doi/10.1145/3366623.3368135`

6. Karim Djemame, Matthew Parker, and Daniel Datsev. "Open-source Serverless Architectures: an Evaluation of Apache OpenWhisk." In *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*, pp. 329–335, 2020. Available at: `https://ieeexplore.ieee.org/document/9302817`

7. Docker Inc. *Docker Engine API Documentation*. Available at: `https://docs.docker.com/reference/api/engine/`

8. Docker Inc. *Docker SDK for Go Documentation*. Available at: `https://pkg.go.dev/github.com/docker/docker`

9. The Go Authors. *The Go Programming Language Documentation*. Available at: `https://go.dev/doc/`

10. Farhan Javed. *Multi-Tiered Serverless Pre-warming Simulator*. GitHub repository. Available at: `https://github.com/frhanjav/final-project`